
CS 421 --- Unification Activity Solutions

Name	Netid

Please write your name/netid legibly in dark ink. Hand in one copy per team. Do not staple or mangle the corners.

Purpose

Unification is a core component of many programming language related algorithms. It is important to be able to solve unification problems by hand, as well as to be able to specify to the computer how to solve such a problem.

Your objectives:

- Explain the syntax and usage of ϕ as a substitution operator.
- Identify the proper situations for each of the four unification rules and the results.
- Explain the necessity of the occurs-check.
- Implement the unification rules in HASKELL.

Part 1 --- ϕ Day

Time estimate: 10 minutes.

For the following table, let $\phi = \{x \mapsto 10, y \mapsto 2\}$

Formula	Result
$\phi(\{(x, y)\})$	$\{(10, 2)\}$
$\phi(\{(a, x), (y, z)\})$	$\{(a, 10), (2, z)\}$
$\phi[x \mapsto z](\{(x, y)\})$	$\{(z, 2)\}$
$\phi[z \mapsto 5](\{(a, x), (x, z)\})$	$\{(a, 10), (10, 5)\}$
$\phi[z \mapsto 5][y \mapsto 20](\{(a, x), (y, z)\})$	$\{(a, 10), (20, 5)\}$

Problem 1) As a team, describe the behavior of ϕ .

- If there is a mapping $x \mapsto y$ in ϕ , how many times will x be replaced in ϕ 's argument? **Every occurrence of x will be replaced.**
- If there is a variable x that has no mapping in ϕ , what happens to the occurrences of x in ϕ 's argument? **Nothing, it is returned as is.**
- If there is a mapping $x \mapsto y$ in ϕ , and we call the function $\phi[x \mapsto z]$, on a term x , which mapping wins? **The right-most one, $x \mapsto z$.**

Problem 2) Now, solve these formulas. Let $\phi = \{x \mapsto a, y \mapsto b\}$

Formula	Result
$\phi(\{(x, y)\})$	$\{(a, b)\}$
$\phi(\{(a, x), (y, z)\})$	$\{(a, a), (b, z)\}$
$\phi[x \mapsto z](\{(x, y)\})$	$\{(z, b)\}$
$\phi[z \mapsto x](\{(a, x), (y, z)\})$	$\{(a, a), (b, x)\}$
$\phi[z \mapsto x][y \mapsto c](\{(a, x), (y, z)\})$	

Note that only one substitution is made. If $\phi = \{x \mapsto y, y \mapsto z\}$, then we get $\phi(x) = y$, not z .

Part 2 --- The Rules

Time estimate: 10 minutes

Given a constraint set C , we define $unify(C)$ as...

- If C is empty, return the identity solution. $\phi(s) = s$
- Otherwise, let $s = t \in C$ and $C' = C \setminus \{s = t\}$.

Delete If s and t are identical then $unify(C')$

Orient If t is a variable and s is not, $unify(\{t = s\} \cup C')$.

Decompose If P is a constructor, $s = P(s_1, \dots, s_n)$ and $t = P(t_1, \dots, t_n)$ then $unify(C' \cup \{s_1 = t_1, \dots, s_n = t_n\})$.

Eliminate If s is a variable, and s does not occur in t , substitute s with t in C' to get C'' . Then let $\phi = unify(C'')$ and return $\phi[s \mapsto \phi(t)]$.

Problem

$unify(\{g(\alpha, a) = g(b, \beta), h(\gamma, \gamma) = h(f(\alpha), \gamma)\})$
 $unify(\{f(\alpha, \alpha) = f(\alpha, \alpha), h(\beta, g(\gamma)) = h(y, \delta)\})$
 $unify(\{f(\alpha) = \delta, g(\alpha) = g(\beta), h(\gamma, x) = h(\beta, \alpha)\})$

Step

Result

Decompose $unify(\{h(\gamma, \gamma) = h(f(\alpha), \gamma), \alpha = b, a = \beta\})$
Delete $unify(\{h(\beta, g(\gamma)) = h(y, \delta)\})$
Orient
 $unify(\{\delta = f(\alpha), g(\alpha) = g(\beta), h(\gamma, x) = h(\beta, \alpha)\})$

Problem 3) The Eliminate rule rewrites ϕ to $\phi[s \mapsto \phi(t)]$. Why can't we just rewrite to $\phi[s \mapsto t]$ instead?

If t has variables in it, we will want those substituted as well.

Problem 4) In Haskell, function calls like `zipWith xx yy` will truncate the longer of xx and yy if they are not the same size. The decompose rule doesn't do this. Why not?

The goal of `zipWith` is to combine data; the goal of unification is to check for compatibility.

Problem 5) Solve the following unification problem, in the order specified above. Label the rule you use for each step.

$$\begin{aligned}
 & unify(\{f(\alpha) = f(x), g(\alpha) = g(\beta), h(\gamma, x) = h(\beta, \alpha)\}) \\
 &= unify(\{\alpha = x, g(\alpha) = g(\beta), h(\gamma, x) = h(\beta, \alpha)\}) \\
 &= \{\alpha = x\} \cup unify(\{g(x) = g(\beta), h(\gamma, x) = h(\beta, x)\}) \\
 &= \{\alpha = x\} \cup unify(\{x = \beta, h(\gamma, x) = h(\beta, x)\}) \\
 &= \{\alpha = x\} \cup unify(\{\beta = x, h(\gamma, x) = h(\beta, x)\}) \\
 &= \{\alpha = x, \beta = x\} \cup unify(\{h(\gamma, x) = h(x, x)\}) \\
 &= \{\alpha = x, \beta = x\} \cup unify(\{\gamma = x, x = x\}) \\
 &= \{\alpha = x, \beta = x, \gamma = x\} \cup unify(\{x = x\}) \\
 &= \{\alpha = x, \beta = x, \gamma = x\}
 \end{aligned}$$

Part 3 --- It Never Occurred to Me

Problem 6) What happens when we try to solve this?

$$\text{unify}(\{f(\alpha) = f(f(\alpha))\})$$

We get an infinitely large solution.

Problem 7) Consider this HASKELL code. What is its type?

```
o foo a = [foo a]
```

We get an infinite type error.

Part 4 --- Show me the Code

This will be put in your repository.